LAPORAN TUGAS PRAKTIKUM
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK
MODUL P10
OBSERVABILITAS SISTEM, STRUCTURED LOGGING, DAN CONTAINER HEALTHCHECKS



Disusun oleh:
Pralambang Bhayangkara Yhudanto
NIM: 24/536223/SV/24414
Kelas: A1

Dosen Pengampu:
Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Gambar **ii**

Daftar Tabel **iii**

1 Tujuan Praktikum **1**

2 Dasar Teori **2**

2.1 Konsep Observabilitas dalam Sistem Terkontainerisasi 2

2.2 Pencatatan Aliran Log (*Log Streaming*) ke STDOUT dan Rotasi Log 2

2.3 Log Terstruktur (*Structured Logging*) dan Kebijakan Zero-PII 3

2.4 Probe Kesehatan Kontainer (*Container Healthchecks*) 3

3 Metodologi dan Langkah Pengerjaan **4**

3.1 Arsitektur Perangkat Lunak dan Repositori 4

3.2 Tahap 1: Inisialisasi Branch Git dan Audit Lingkungan 4

3.3 Tahap 2: Pengalihan Log ke STDOUT dan Pembatasan Ukuran Log 4

3.4 Tahap 3: Implementasi Tiga Alur Structured Logging (Zero-PII) 5

3.5 Tahap 4: Implementasi Endpoint Observabilitas /health Mendalam 5

3.6 Tahap 5: Konfigurasi Healthcheck Docker Compose dan Chaos Test 6

3.7 Tahap 6: Integrasi Git, Branching Hygiene, dan Pull Request 6

4 Hasil Pengujian dan Analisis Observabilitas **7**

4.1 Hasil Inisialisasi Cabang Git (Tahap 1) 7

4.2 Hasil Pengalihan Log ke STDOUT dan Batasan Rotasi Log (Tahap 2) 7

4.3 Hasil Implementasi Log Terstruktur Berkonteks (Tahap 3) 8

4.4 Hasil Pengujian Endpoint Observabilitas /health (Tahap 4) 9

4.5 Hasil Pengujian Ketahanan Kontainer Dua Arah (Tahap 5) 9

4.6 Hasil Integrasi Git dan Pull Request (Tahap 6) 11

5 Kesimpulan **13**

Daftar Gambar

4.1 Bukti Inisialisasi Feature Branch P10 (`git status` dan `git branch`) 7
4.2 Bukti Konfigurasi Log Driver Compose dan Tangkapan Log Laravel di Terminal Host 8
4.3 Bukti Output Terminal Tiga Alur Log Terstruktur Berkonteks (INFO, WARNING, ERROR) 8
4.4 Respons HTTP 200 OK dan Payload JSON Validasi Nyata Endpoint `/health` . 9
4.5 Status Layanan Docker Compose Kondisi Normal Berstatus `(healthy)` 10
4.6 Simulasi Pemadaman Sengaja Kontainer Basis Data (`docker compose stop db`) 10
4.7 Deteksi Otomatis Gangguan: Kolom Status Backend Berubah Menjadi `(unhealthy)` 10
4.8 Pengaktifan Kembali Kontainer Basis Data (`docker compose start db`) 10
4.9 Pemulihan Otomatis (*Self-Healing*): Status Backend Kembali Menjadi `(healthy)` 10
4.10 Verifikasi Endpoint `/health` Kembali Merespons HTTP 200 OK Pasca Pemulihan 11
4.11 Bukti Penggabungan Pull Request Resmi di GitHub ke Cabang `dev` 11
4.12 Bukti Riwayat Atomic Commits Berstandar Conventional Commits pada Pull Request 12

Daftar Tabel

4.1 Analisis dan Alasan Pemilihan Alur Structured Logging 9

Bab 1. Tujuan Praktikum

Berdasarkan modul pelaksanaan tugas praktikum Konstruksi dan Evolusi Perangkat Lunak (KEPL) Modul P10 mengenai *Observabilitas Sistem, Structured Logging, dan Container Healthchecks*, tujuan pembelajaran dan luaran teknis yang hendak dicapai adalah sebagai berikut:

1. Mahasiswa mampu menerapkan konsep observabilitas sistem modern pada container Docker dengan mengalihkan aliran pencatatan (*logging stream*) dari file lokal internal (`laravel.log`) menuju standard output (`php://stdout`) container.

2. Mahasiswa mampu mengonfigurasi batas ukuran berkas log dan rotasi log container (`max-size` dan `max-file`) pada Docker Compose untuk mencegah saturasi kapasitas disk server.

3. Mahasiswa mampu merancang dan mengimplementasikan pencatatan log terstruktur dan berkonteks (*Contextual Structured Logging*) pada tiga alur krusial aplikasi (tingkat `INFO`, `WARNING`, dan `ERROR`) dengan mematuhi kebijakan perlindungan privasi tanpa kebocoran data sensitif (*Zero-PII Policy*).

4. Mahasiswa mampu membangun endpoint inspeksi kesehatan mandiri (`GET /health`) yang secara aktif memvalidasi status koneksi riil ke basis data (MySQL) dan cache engine.

5. Mahasiswa mampu mengonfigurasi probe `healthcheck` native pada Docker Compose dengan parameter *interval*, *timeout*, *retries*, dan *start_period*, serta membuktikan ketahanan sistem dua arah (*Chaos/Outage Resilience Test*) saat subsistem basis data mengalami gangguan dan saat pemulihan otomatis (*self-healing*).

6. Mahasiswa mampu menerapkan disiplin rekayasa perangkat lunak melalui *Branching Hygiene* Git dan *Conventional Commits* dalam alur kerja kolaboratif berbasis Pull Request.

Bab 2. Dasar Teori

2.1 Konsep Observabilitas dalam Sistem Terkontainerisasi

Observabilitas (*observability*) adalah kemampuan untuk memahami kondisi internal suatu sistem komputasi perangkat lunak berdasarkan luaran eksternal yang dihasilkannya. Dalam arsitektur sistem berbasis kontainer (seperti Docker), aplikasi tidak lagi berjalan pada server bare-metal statis, melainkan pada lingkungan yang bersifat efemeral (*ephemeral*) dan dinamis. Tiga pilar utama observabilitas modern mencakup:

- ***Logs*:** Catatan diskrit berbasis waktu yang mendokumentasikan kejadian spesifik dalam siklus hidup aplikasi.
- ***Metrics*:** Nilai numerik agregat terukur yang merepresentasikan utilisasi sumber daya komputasi atau throughput transaksi.
- ***Traces*:** Jejak siklus hidup permohonan (*request lifecycle*) dari awal masuk hingga selesai melintasi berbagai layanan.

2.2 Pencatatan Aliran Log (*Log Streaming*) ke STDOUT dan Rotasi Log

Berdasarkan prinsip metodologi arsitektur modern *Twelve-Factor App* (khususnya faktor ke-11: *Logs*), aplikasi tidak boleh dibebani tanggung jawab untuk mengelola atau menyimpan berkas log sendiri ke dalam filesystem lokal kontainer. Apabila aplikasi mencatat log ke dalam direktori internal (misalnya `storage/logs/laravel.log`), berkas tersebut akan hilang ketika container di-restart atau di-deploy ulang, serta membebani layer I/O container. Oleh karena itu, seluruh kejadian log harus diperlakukan sebagai aliran kejadian (*event stream*) yang disemburkan langsung ke `stdout` atau `stderr`.

Docker daemon menangkap aliran `stdout` dan `stderr` tersebut melalui *logging driver* (secara default `json-file`). Namun, penulisan log secara terus-menerus tanpa pembatasan dapat menyebabkan masalah kepenuhan disk host (*disk exhaustion*). Melalui konfigurasi opsi log rotation:

- `max-size`: Menentukan batas ukuran maksimal satu file berkas log sebelum dilakukan rotasi (misalnya `10m`).
- `max-file`: Menentukan jumlah maksimum file arsip yang disimpan sebelum file terlama dihapus secara otomatis (misalnya `3`).

2.3 Log Terstruktur (*Structured Logging*) dan Kebijakan Zero-PII

Pencatatan log tradisional umumnya berupa teks bebas (*unstructured text*) yang sulit diproses dan diurai (*parsing*) oleh sistem agregator log otomatis. Sebaliknya, *Structured Logging* menyajikan informasi dalam format data terstruktur (seperti JSON atau asosiasi kunci-nilai) yang memuat konteks kejadian secara komprehensif:

- *Actor*: Identitas publik pengirim permintaan (misal: `user_id`, `ip_address`, `user_agent`).
- *Action*: Operasi spesifik yang sedang dieksekusi (misal: `buku.create`).
- *Reason / Resource*: Entitas target atau alasan kegagalan validasi / query.
- *Timestamp*: Waktu terjadinya transaksi dalam standar ISO-8601.

Dalam implementasi log komersial dan akademik, kepatuhan terhadap ***Zero Personally Identifiable Information*** **(Zero-PII Policy)** merupakan syarat absolut. Data sensitif seperti kata sandi (password), hash sandi, API key, Bearer token otorisasi, dan nomor kartu identitas dilarang keras dicatat ke dalam log. Data identitas pengguna wajib disanitasi atau disamarkan (*masked*), misalnya `pr**********@gmail.com`.

2.4 Probe Kesehatan Kontainer (*Container Healthchecks*)

Mekanisme *Healthcheck* pada Docker Compose memungkinkan mesin orkestrasi mengetahui apakah aplikasi di dalam container benar-benar siap dan mampu melayani permintaan bisnis (*readiness*), bukan sekadar mendeteksi apakah proses PID masih berjalan. Docker menyediakan empat parameter utama:

1. `test`: Perintah pengujian berkala yang dijalankan di dalam kontainer (misalnya melakukan request HTTP via `wget` ke endpoint kesehatan).
2. `interval`: Frekuensi selang waktu antara eksekusi probe kesehatan (misalnya setiap 10 detik).
3. `timeout`: Batas durasi maksimum menunggu respons probe sebelum dianggap gagal (misalnya 5 detik).
4. `retries`: Ambang batas jumlah kegagalan berturut-turut sebelum status kontainer diubah menjadi `unhealthy` (misalnya 3 kali).
5. `start_period`: Periode toleransi awal untuk memberi waktu inisialisasi aplikasi tanpa mencatat status unhealthy (misalnya 15 detik).

Bab 3. Metodologi dan Langkah Pengerjaan

Praktikum P10 dieksekusi secara metodis dan disiplin tinggi pada repositori proyek `uts-pl-536223`. Seluruh tahap pekerjaan dilakukan secara bertahap dengan pencabangan fitur Git, pengujian fungsional, dan pencatatan riwayat commit berstandar *Conventional Commits*.

3.1 Arsitektur Perangkat Lunak dan Repositori

Sistem yang dikembangkan memiliki arsitektur terisolasi berbasis Docker Compose dengan rincian layanan sebagai berikut:

- **Backend REST API**: Framework Laravel 12 dengan runtime PHP 8.4 Alpine CLI pada kontainer `uts_pl_backend` (Port host 8000).
- **Basis Data Relasional**: MySQL 8.0 pada kontainer `uts_pl_db` dengan persistent volume `mysql_data`.
- **Frontend SPA**: Vue.js 3 Single Page Application yang disajikan melalui web server Nginx pada kontainer `uts_pl_frontend` (Port host 80).
- **Tautan Repositori GitHub Publik**: https://github.com/bhayudha/uts-pl-536223

3.2 Tahap 1: Inisialisasi Branch Git dan Audit Lingkungan

Tahap awal dimulai dengan memastikan kondisi repositori lokal sinkron terhadap cabang pengembangan utama (`dev`), kemudian membuat cabang fitur baru khusus untuk modul observabilitas:

Listing 3.1: Perintah Inisialisasi Feature Branch P10

```
git checkout dev
git pull origin dev
git checkout -b feature/p10-logging-healthcheck
```

3.3 Tahap 2: Pengalihan Log ke STDOUT dan Pembatasan Ukuran Log

Pada tahap ini, berkas konfigurasi logging Laravel `backend/config/logging.php` dimodifikasi untuk mendaftarkan driver `stdout` Monolog menggunakan `StreamHandler` ke aliran

`php://stdout`. Selain itu, variabel lingkungan `LOG_CHANNEL` pada berkas `backend/.env` dan `backend/.env.example` diubah menjadi `stdout`.

Pada berkas `docker-compose.yml`, konfigurasi logging ditambahkan pada service `db` dan `backend`:

Listing 3.2: Konfigurasi Rotasi Log pada docker-compose.yml

```yaml
services:
  db:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
  backend:
    environment:
      LOG_CHANNEL: stdout
      LOG_LEVEL: debug
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

3.4 Tahap 3: Implementasi Tiga Alur Structured Logging (Zero-PII)

Pencatatan log terstruktur diterapkan pada Controller katalog buku (`backend/app/Http/Controllers/Bu` untuk memantau siklus hidup penyimpanan data:

1. **Tingkat `INFO`**: Dicatat saat penambahan data buku berhasil, merekam ID pengguna, email yang disamarkan (*masked*), IP address, aksi `buku.create`, serta identitas buku.
2. **Tingkat `WARNING`**: Dicatat saat validasi form gagal, merekam aktor, daftar atribut yang tidak valid (`invalid_fields`), dan user agent tanpa merekam isi data sensitif.
3. **Tingkat `ERROR`**: Dicatat dalam blok `try-catch` saat transaksi query basis data mengalami kegagalan/outage, merekam kode kesalahan SQL/PDO (`error_code`) tanpa mengekspos kredensial koneksi.

3.5 Tahap 4: Implementasi Endpoint Observabilitas /health Mendalam

Endpoint HTTP observabilitas dibuat secara mandiri pada Controller baru `backend/app/Http/Controlle` dan didaftarkan sebagai rute publik pada `backend/routes/web.php`. Endpoint ini melakukan:

- Pengujian nyata koneksi basis data MySQL menggunakan `DB::connection()->getPdo()` dan query `SELECT 1`.
- Pengujian baca-tulis engine Cache Laravel menggunakan operasi `put`, `get`, dan `forget`.
- Menghasilkan kode status HTTP 200 OK jika seluruh subsistem normal, atau HTTP 503 Service Unavailable jika terdapat dependensi yang gagal.

3.6 Tahap 5: Konfigurasi Healthcheck Docker Compose dan Chaos Test

Instrumen healthcheck diintegrasikan ke dalam service `backend` pada `docker-compose.yml`:

Listing 3.3: Konfigurasi Healthcheck pada docker-compose.yml

```
healthcheck:
  test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://127.0.0.1:8000/health || exit 1"
      ]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 15s
```

Pengujian ketahanan dua arah (*two-way outage resilience*) dieksekusi melalui skenario:

- **Arah 1 (Normal/Healthy)**: Memvalidasi kontainer berstatus `(healthy)` saat sistem berjalan normal.
- **Arah 2 (Gangguan/Unhealthy)**: Mematikan kontainer basis data (`docker compose stop db`) secara sengaja, memverifikasi status berubah menjadi `(unhealthy)`, endpoint membalas HTTP 503, dan log error tercatat.
- **Arah 3 (Pemulihan/Self-Healing)**: Menyalakan kembali basis data (`docker compose start db`) dan membuktikan kontainer otomatis kembali berstatus `(healthy)` dan HTTP 200 OK.

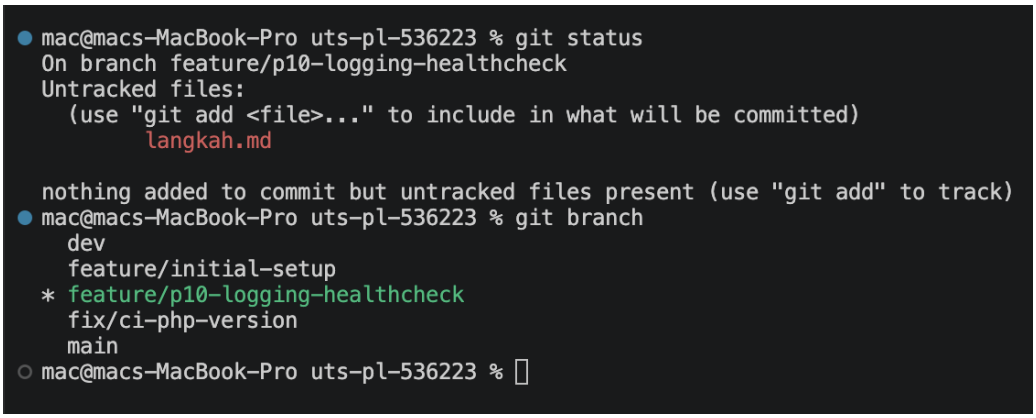
3.7 Tahap 6: Integrasi Git, Branching Hygiene, dan Pull Request

Branch fitur diunggah ke repositori GitHub, kemudian digabungkan ke cabang `dev` melalui mekanisme Pull Request resmi (`PR #4`), dilanjutkan dengan penggabungan rilis ke cabang utama `main`.

Bab 4. Hasil Pengujian dan Analisis Observabilitas

4.1 Hasil Inisialisasi Cabang Git (Tahap 1)

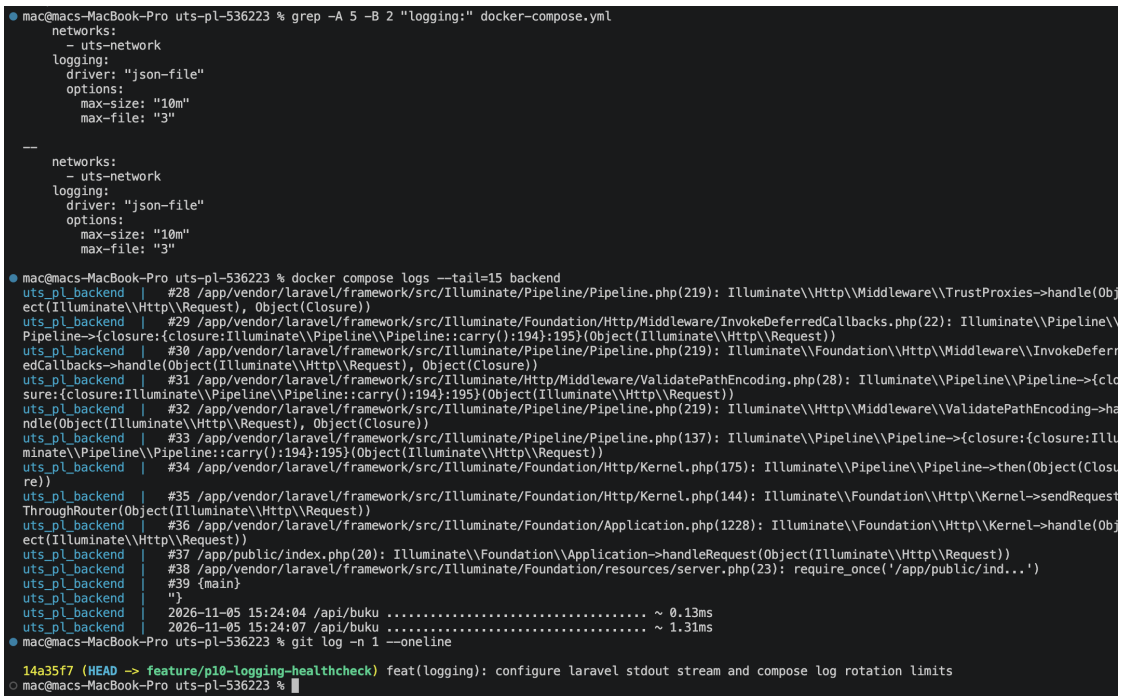
Pengujian tahap pertama memvalidasi kebersihan repositori dan kepatuhan terhadap isolasi cabang fitur. Gambar 4.1 membuktikan bahwa pengerjaan tugas P10 dimulai dari cabang terisolasi `feature/p10-logging-healthcheck`.



Gambar 4.1: Bukti Inisialisasi Feature Branch P10 (`git status` dan `git branch`)

4.2 Hasil Pengalihan Log ke STDOUT dan Batasan Rotasi Log (Tahap 2)

Konfigurasi pembatasan rotasi log pada `docker-compose.yml` diverifikasi menggunakan perintah `grep`, sementara aliran log internal Laravel dan server backend dibuktikan langsung tertangkap pada terminal host melalui perintah `docker compose logs backend`. Hasil verifikasi ditampilkan pada Gambar 4.2.

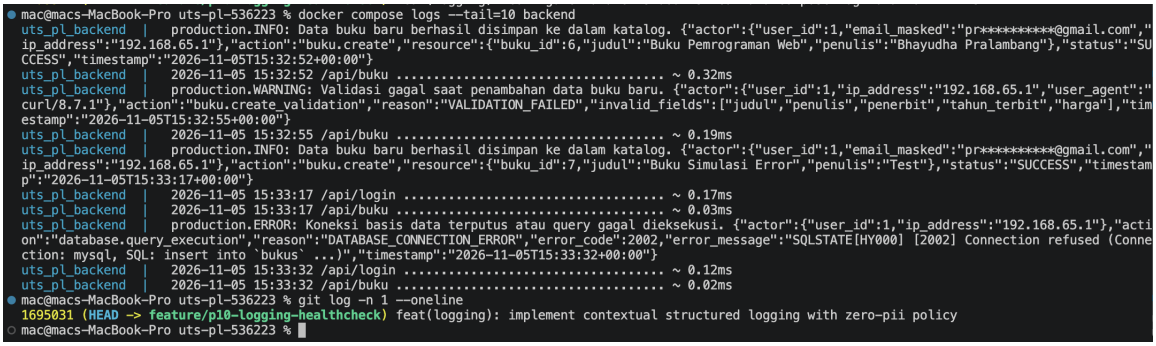


Gambar 4.2: Bukti Konfigurasi Log Driver Compose dan Tangkapan Log Laravel di Terminal Host

Dapat dianalisis bahwa dengan pengalihan ini, log aplikasi tidak lagi mengendap di filesystem kontainer, melainkan langsung dikonsumsi oleh Docker logging engine dengan batas rotasi file sebesar 10MB sebanyak maksimal 3 file arsip.

4.3 Hasil Implementasi Log Terstruktur Berkonteks (Tahap 3)

Pengujian terhadap tiga alur logging krusial diverifikasi melalui panggilan REST API. Output log yang dihasilkan oleh sistem ditampilkan pada Gambar 4.3.



Gambar 4.3: Bukti Output Terminal Tiga Alur Log Terstruktur Berkonteks (INFO, WARNING, ERROR)

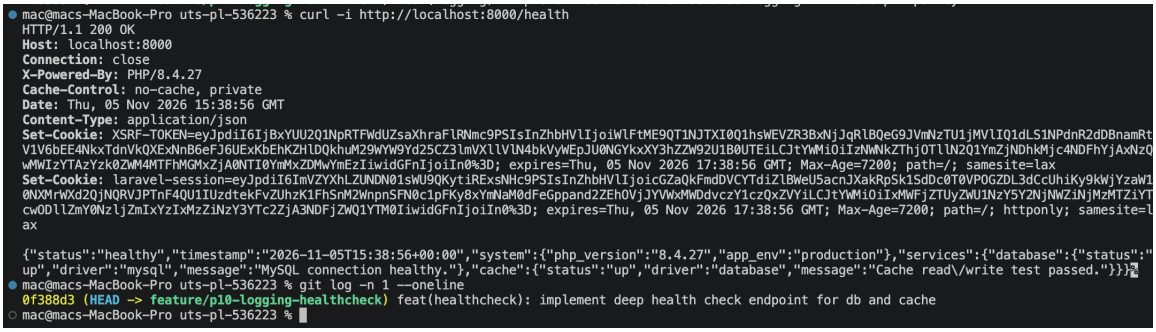
Tabel 4.1 merinci analisis teknis dan alasan rasional pemilihan ketiga alur logging tersebut dalam siklus hidup operasional aplikasi.

Tabel 4.1: Analisis dan Alasan Pemilihan Alur Structured Logging

Tingkat	Alur / Endpoint	Konteks yang Dicatat	Alasan Teknis Pemilihan Alur
INFO	Penambahan Buku Baru (`POST /api/buku`)	`user_id`, `email_masked`, `ip_address`, `action: buku.create`, `buku_id`, `judul`, status `SUCCESS`.	Memberikan jejak audit (*audit trail*) yang jelas atas setiap mutasi data katalog yang berhasil tanpa mencatat data rahasia/PII pengguna.
WARNING	Validasi Form Gagal (`POST /api/buku`)	`user_id`, `ip_address`, `user_agent`, `action`, `reason: VALIDATION_FAILED`, daftar `invalid_fields`.	Mendeteksi anomali transmisi payload, potensi manipulasi parameter (*tampering*), atau ketidaksesuaian antarmuka frontend tanpa mengekspos isi nilai masukan.
ERROR	Kegagalan Query Basis Data / Outage	`user_id`, `ip_address`, `action`, `reason: DATABASE_ERROR`, `error_code`, `error_message`.	Menangkap insiden kegagalan infrastruktur secara real-time guna memangkas durasi resolusi insiden (*Mean Time to Resolution*) tanpa membocorkan kredensial koneksi DB.

4.4 Hasil Pengujian Endpoint Observabilitas /health (Tahap 4)

Endpoint observabilitas kesehatan `GET /health` diuji menggunakan perintah `curl -i http://localhost` Hasil pengujian disajikan pada Gambar 4.4.



Gambar 4.4: Respons HTTP 200 OK dan Payload JSON Validasi Nyata Endpoint `/health`

Respons membuktikan status HTTP 200 OK dengan payload JSON terstruktur yang memvalidasi bahwa koneksi basis data MySQL berstatus `up` dan engine Cache berstatus `up`.

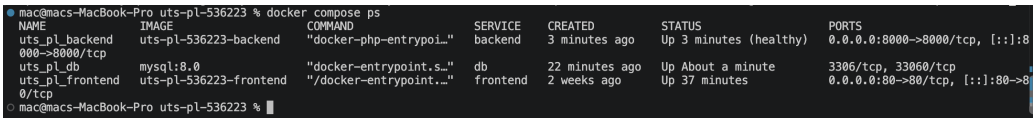
4.5 Hasil Pengujian Ketahanan Kontainer Dua Arah (Tahap 5)

Pengujian ketahanan dua arah (*Chaos/Outage Test*) membuktikan keandalan otomatisasi healthcheck:

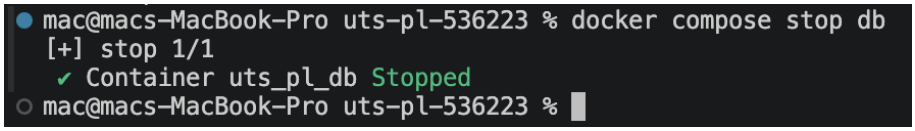
1. **Kondisi Sehat Awal (*Healthy*)**: Kontainer backend beroperasi dengan status terverifikasi sehat, sebagaimana ditunjukkan pada Gambar 4.5.

2. **Kondisi Sakit / Gangguan (*Unhealthy*)**: Layanan basis data dihentikan secara sengaja (`docker compose stop db`) seperti pada Gambar 4.6. Setelah melewati batas ambang retries, status kontainer backend secara otomatis berubah menjadi `Up (unhealthy)` pada Gambar 4.7.

3. **Kondisi Pemulihan (*Self-Healing*)**: Layanan basis data dinyalakan kembali (`docker compose start db`) seperti pada Gambar 4.8. Dalam hitungan detik, kontainer backend secara otomatis kembali pulih menjadi berstatus `Up (healthy)` pada Gambar 4.9, dan endpoint `/health` kembali membalas dengan status HTTP 200 OK pada Gambar 4.10.



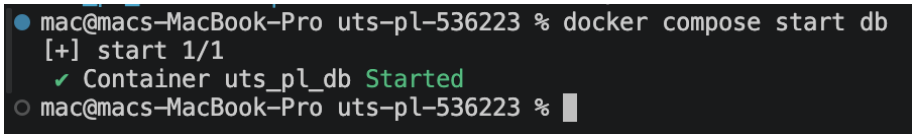
Gambar 4.5: Status Layanan Docker Compose Kondisi Normal Berstatus `(healthy)`



Gambar 4.6: Simulasi Pemadaman Sengaja Kontainer Basis Data (`docker compose stop db`)



Gambar 4.7: Deteksi Otomatis Gangguan: Kolom Status Backend Berubah Menjadi `(unhealthy)`



Gambar 4.8: Pengaktifan Kembali Kontainer Basis Data (`docker compose start db`)



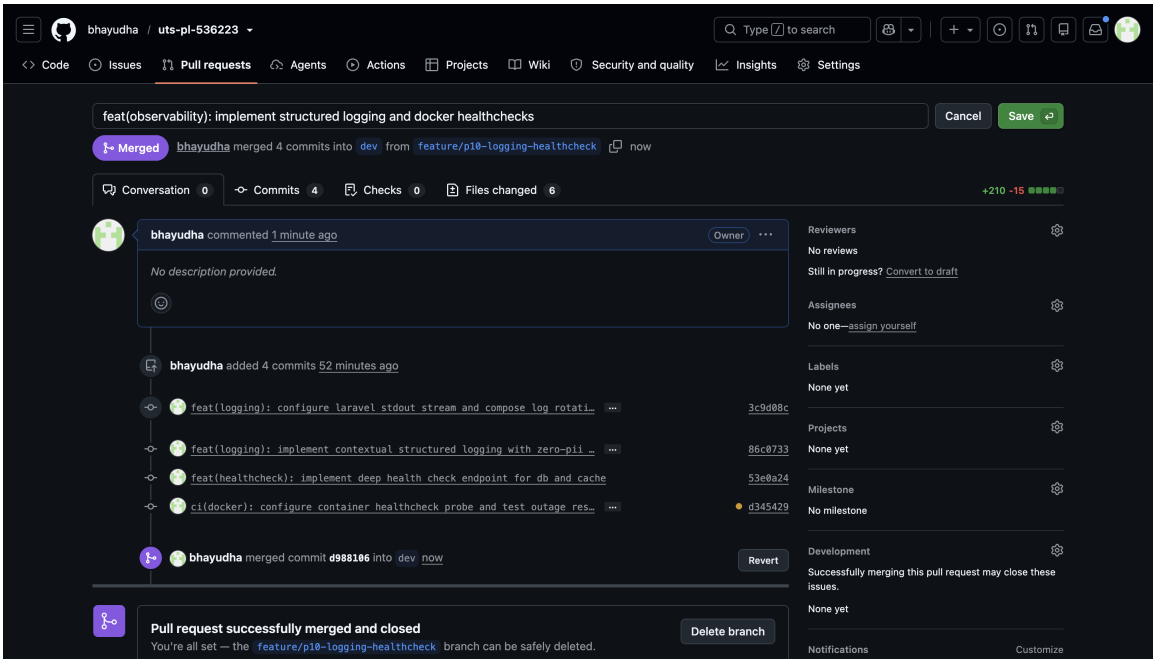
Gambar 4.9: Pemulihan Otomatis (*Self-Healing*): Status Backend Kembali Menjadi `(healthy)`



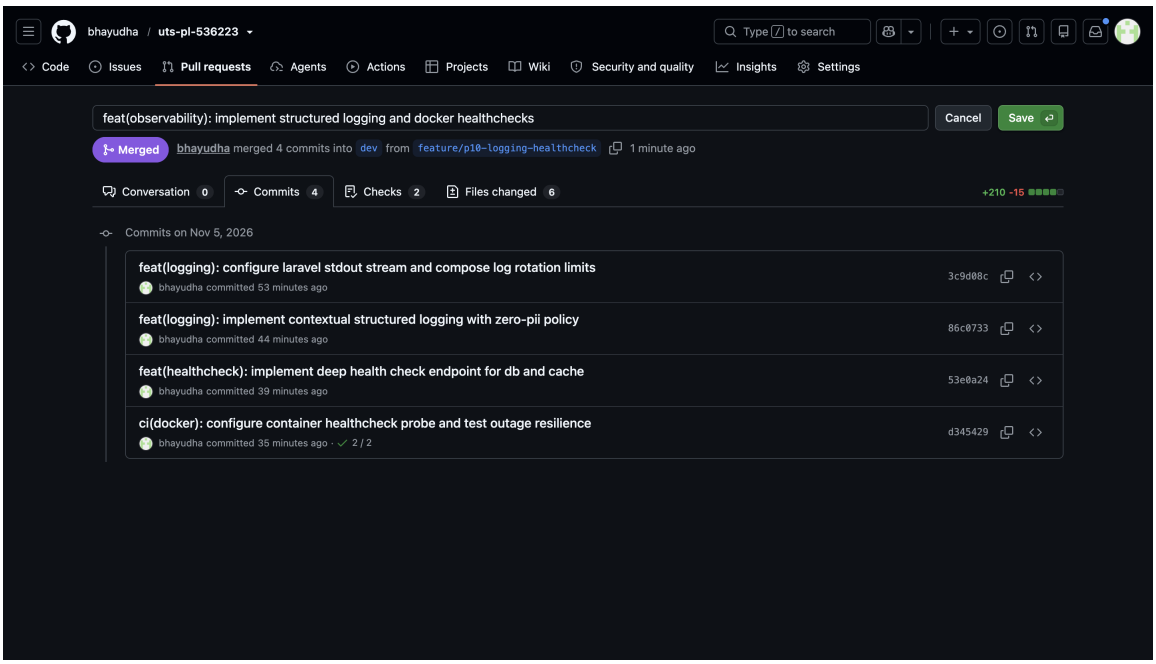
Gambar 4.10: Verifikasi Endpoint `/health` Kembali Merespons HTTP 200 OK Pasca Pemulihan

4.6 Hasil Integrasi Git dan Pull Request (Tahap 6)

Kepatuhan terhadap standar kebersihan Git dibuktikan melalui riwayat Pull Request di GitHub. Gambar 4.11 memperlihatkan penggabungan fitur ke cabang `dev` yang berstatus *Merged*. Selain itu, Gambar 4.12 menunjukkan empat commit atomik berurutan yang mematuhi konvensi *Conventional Commits*.



Gambar 4.11: Bukti Penggabungan Pull Request Resmi di GitHub ke Cabang `dev`



Gambar 4.12: Bukti Riwayat Atomic Commits Berstandar Conventional Commits pada Pull Request

Bab 5. Kesimpulan

Berdasarkan perancangan, implementasi, dan serangkaian pengujian yang telah dilakukan pada tugas praktikum Konstruksi dan Evolusi Perangkat Lunak Modul P10, dapat ditarik beberapa kesimpulan penting sebagai berikut:

1. Pengalihan aliran log Laravel langsung ke standard output (`php://stdout`) memenuhi kaidah *Twelve-Factor App*, memungkinkan Docker daemon menangkap seluruh kejadian sistem secara real-time tanpa membebani disk internal kontainer.

2. Konfigurasi pembatasan rotasi log pada Docker Compose (`max-size: 10m` dan `max-file: 3`) terbukti efektif mengamankan host dari ancaman kepenuhan ruang simpan akibat akumulasi volume log tak terkendali.

3. Penerapan *Contextual Structured Logging* pada tiga alur vital (`INFO`, `WARNING`, `ERROR`) berhasil meningkatkan visibilitas operasional sistem dengan menyajikan konteks lengkap (*who, what, why, when*) secara terstruktur dengan kepatuhan penuh terhadap kebijakan *Zero-PII*.

4. Endpoint observabilitas mandiri `GET /health` mampu melakukan pengujian aktif terhadap dependensi vital (MySQL dan Cache), memberikan pembedaan tegas antara status siap melayani (HTTP 200) dan status degradasi layanan (HTTP 503).

5. Konfigurasi probe `healthcheck` native Docker Compose berhasil memvalidasi ketahanan dua arah secara tangguh, di mana orkestrator kontainer mampu mendeteksi pemadaman database secara otomatis (*healthy* $\rightarrow$ *unhealthy*) dan memulihkan status operasional layanan pasca perbaikan (*unhealthy* $\rightarrow$ *healthy*).

6. Penerapan alur kerja Git berbasis *Branching Hygiene* dan *Conventional Commits* melalui Pull Request menjamin keterlacakan evolusi kode sumber dan kerapian riwayat pengembangan pada tingkat industri profesional.